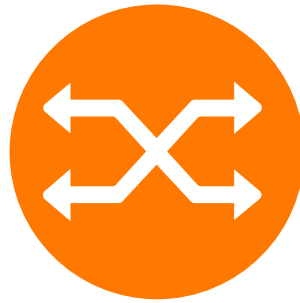


Orange Research

Shape the future and value innovation

Challenge EURO/ROADEF 2026

Keep the flow!



Challenge Rules

Challenge Organizers

Amal Benhamiche, Yannick Carlinet, Morgan Chopin, Eric Gourdin, Nancy Perrot



Contents

1	General rules	1
2	Organization	1
2.1	Roadmap	1
2.2	Prizes	1
2.3	Datasets	2
2.4	Evaluation	2
2.5	Ranking method	3
2.5.1	Definition	3
2.5.2	Example	3
2.6	Submission	3
2.7	Computing and software limitations	4
2.8	Format of the submitted program	4
2.9	Variability	5
2.10	Available solvers	5
2.11	Provided tools	5
2.11.1	Checker	5
2.11.2	Networktools	6
2.12	Contact information	6
3	Discussion and updates	6
4	Intellectual property	6
A	Examples of Submissions	7
A.1	Python Program	7
A.2	C++ Program with the Networktools library	8
B	How to test your program before submission	9

1 General rules

Introduction The EURO/ROADEF 2026 Challenge is dedicated to a problem related to Segment Routing, entitled “Adaptive Segment Routing – Keep the Flow!”. The problem description, benchmark instances, and solution checker are provided by Orange. Participants compete to design and submit the most effective algorithmic solutions for solving the proposed problem. All official information regarding the challenge is made available on the challenge website. Participants are invited to submit feedback, including questions, concerns, or the reporting of potential errors, to the organizing committee. For this purpose, a dedicated forum on the challenge website and a specific email address are provided (see Section 2.12). The organizing committee endeavors to respond to participants’ inquiries as accurately and promptly as possible.

General rules

- The following categories of individuals are expressly excluded from participation in the challenge:
 - Employees of Orange
 - Interns at Orange
 - Members of the organizing committee
- The Organizing Committee shall not disclose any information relating to the challenge to any third party, except for information that is made publicly available to all participants.

Team composition The rules governing team composition are defined as follows:

- A team may consist of any number of individuals.
- No individual shall be a member of more than one team.
- Each team is authorized to submit one and only one program to solve the problem.
- A junior team shall be composed exclusively of student members.
- A member shall be deemed a student if they are enrolled in an undergraduate, graduate, or PhD program as of December 31, 2026. If the PhD has been defended prior to this date, the individual shall not be considered a student for the purposes of the challenge.

2 Organization

This section describes the general steps of the challenge.

2.1 Roadmap

The challenge starts on February 25th, 2026 and results will be announced at the IFORS 2026 conference, the ROADEF 2027 conference and the ROADEF 2028 for the scientific prize (see Figure 1)

The teams can participate to the sprint and qualification phase independently to each other. After the qualification phase, only qualified teams selected by the organizers (see Section 2.5) will be allowed to participate to the final phase.

2.2 Prizes

The total amount of prizes is 40k€, divided in several categories as explained at <https://www.roadef.org/challenge/2026/en/prix.php>

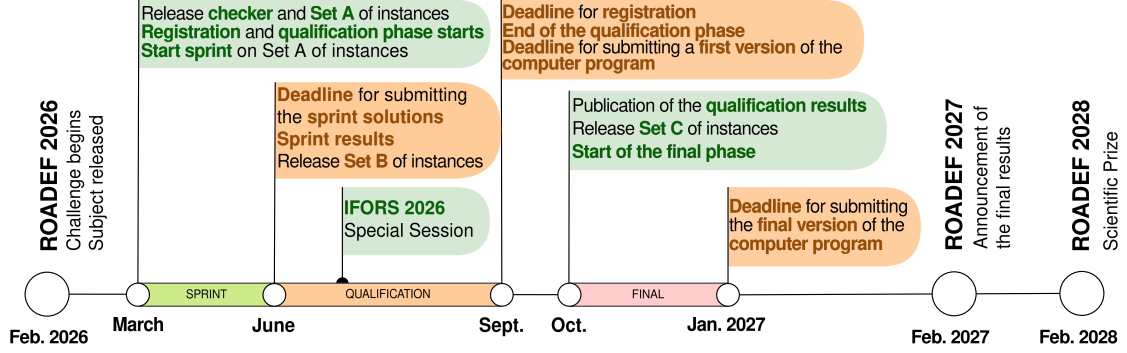


Figure 1: ROADEF 2026 Challenge roadmap. The deadlines can be found on the challenge website <https://roadef.org/challenge/2026/en/calendrier.php>

2.3 Datasets

Four datasets composed of industrial problem instances are provided to participants during the challenge:

- Dataset A of 20 instances at the beginning of the challenge.
- Dataset B of 12 instances available after the end of the sprint phase, that can be used by candidates for testing their program.
- Dataset X of 12 hidden instances, similar to set B, that will be used to rank the candidates for the qualification phase and available after the end of the phase.
- Dataset C of 12 instances available at the start of the final phase, that can be used by candidates for testing their program.
- Dataset Y of 12 hidden instances, similar to set C, that will be used to rank the candidates for the final phase and available after the end of the challenge.

2.4 Evaluation

Sprint phase During this phase, participants are not required to submit any program code or documentation; only solution files corresponding to the proposed instances shall be provided. Upon expiration of the Sprint Phase submission deadline, each team must have submitted its solution files. Dataset A is used as the sole basis for comparing and ranking teams. A prize is awarded to the team ranked first at the conclusion of this phase.

Qualification phase At the deadline of the qualification phase, each team must send its computer program (see Section 2.7) and a short document (2 pages max), that describes the solving method. The dataset X will be used to compare teams. Dataset X will not be disclosed before the evaluation. The time limit for the program execution is 10 minutes per instance in this phase. Teams qualified for the final phase will be selected according to their score (cf. Section 2.5) and the quality of their documentation.

Final phase At the deadline of the final phase, each team must send its computer program (see Section 2.7) and a document (5 pages max), which describes the solving method. The datasets Y will be used to compare teams. Dataset Y will not be disclosed before the evaluation. The computer program of each team will be executed on each instance within the time limit (15 minutes per instance in final phase).

2.5 Ranking method

2.5.1 Definition

At the end of each phase, teams are ranked according to the solutions they provided. The used ranking function is the following. Each team earns points for every instance, depending on their result. We denote by *score* the number of points earned by a team. The winner is the team that maximizes the sum of its scores over all instances.

Solution Comparison. For a given instance, two routing scheme solutions can be compared as follows. They are entered in the checker (cf. Section 2.11.1), with the option ‘--max-decimal-places 6’, which gives in return a vector of loads for all arcs and all timesteps, sorted in decreasing order. The resulting vectors for the two solutions are then compared lexicographically. The solution with a strictly lower resulting vector is said to be strictly better than the other one.

More formally, given two solutions S and S' , we say that S is strictly better than S' , denoted by $S \succ S'$, if, and only if, there exists $k \in \{1, \dots, |A| \times |T|\}$ such that,

$$\begin{cases} \lambda_S(k) < \lambda_{S'}(k) \\ \lambda_S(i) = \lambda_{S'}(i), \forall i < k \end{cases}$$

where T is the input time periods, A the set of input network arcs and $\lambda_X(i)$ denotes the i -th largest load induced by a routing scheme solution X among all arcs and all timesteps.

Let n be the total number of teams and m the total number of instances in a dataset. Let S_{ij} be the solution provided by team $i \in [n]$ for instance $j \in [m]$. We denote by nb_{ij} the number of teams with a solution strictly better than the solution of team i on instance j ,

$$nb_{ij} = |\{k \in [n] : S_{kj} \succ S_{ij}\}|$$

Let R be the maximal score that a team can earn from one instance. R will be set at the start of each phase, according to the number of participants.

Let p_{ij} be the number of points scored (i.e., the score) by team i for the instance j . It is defined as follows,

$$p_{ij} = \begin{cases} \max(0, R - nb_{ij}) & \text{if the solution is feasible} \\ 0 & \text{if the solution is infeasible or missing} \end{cases}$$

The global score of a team, denoted as score_i is defined by,

$$\text{score}_i = \sum_{j=1}^m p_{ij}$$

The winner is the team with the highest score. In case of equality (i.e., several teams with the same highest score), the winner is the team with the highest number of best known solutions, that is the team i where $|\{j \in [m] : nb_{ij} = 0\}|$ is maximum.

2.5.2 Example

Table 1 illustrates how the top team is identified. In this example, we consider $m = 6$ instances of the T -ADAPTIVE SEGMENT ROUTING problem, $n = 7$ teams and $R = 5$. According to table 1(c), Team 5 would be the winner. It has the same score as Team 6 (20), but a higher number of best known solutions (2 vs 0).

2.6 Submission

Before submission, teams must register by completing the form on the challenge website:

<https://roaDEF.org/challenge/2026/en/registration.php>

Instance	1	2	3	4	5	6
Team 1	2	5	2	2	1	3
Team 2	0	0	5	4	6	5
Team 3	1	6	0	0	1	6
Team 4	3	4	6	2	5	4
Team 5	4	2	3	1	0	0
Team 6	6	1	1	5	1	2
Team 7	5	2	3	6	1	0

(a) Values of nb_{ij}

Instance	1	2	3	4	5	6
Team 1	3	0	3	3	4	2
Team 2	5	5	0	1	0	0
Team 3	4	0	5	5	4	0
Team 4	2	1	0	3	0	1
Team 5	1	3	2	4	5	5
Team 6	0	4	4	5	4	3
Team 7	0	3	2	0	4	5

(b) Values of p_{ij}

Team	Score
Team 1	15
Team 2	11
Team 3	18
Team 4	7
Team 5	20
Team 6	20
Team 7	14

(c) Values of $score_i$ Table 1: Example score computation with 6 problem instances, 7 teams and $R = 5$

The solutions (and programs/documents for the qualification/final phases) must be submitted by email at this address:

challenge.roadef2026@orange.com

with the following guidelines:

- The submission must be attached to the email as a zip archive.
- The file name of the archive must be ‘<team.ID>.zip’, where <team.ID> is the id given at subscription (e.g. S42).
- The email must have the subject ‘[ChallengeROADEF2026] [<phase>] <team.ID>’, where <phase> is one of ‘sprint’, ‘qualification’ or ‘final’.

For the sprint phase, there is no ‘run.sh’ script (see Section 2.8), only the solution files. They must be named using the format ‘<instance.name>-srpaths.json’ so they can be directly read by the checker (for instance ‘setA-04-srpaths.json’).

Candidates are able to submit as many times as they wish, only the last email received before the deadline will be considered. Upon submission, candidates will receive a confirmation email that the submission was received. For the sprint phase, it is the responsibility of the candidates to check that the submitted solution files are considered valid by the checker (cf. Section 2.11.1).

For the programs submitted on qualification and final phases, in the case they are too big to be sent by email, candidates are allowed to upload their archive to a transfer file system (such as wetransfer.com for instance). The link to download the archive will then be sent in the submission email.

The archive submitted for the qualification and final phases must follow the guidelines provided in Section 2.8.

2.7 Computing and software limitations

To make a fair comparison of methods developed by different participating teams, each of them has to deliver for qualification and final phases an executable code. This program must take as inputs a problem instance from one of the datasets according to format given in the problem description document. It must return output solution files using standards defined in the problem description document. Programs of candidates will be run on Google Cloud Platform, in VM with 8 CPU (Xeon 2.80GHz), 32 GB of RAM and Ubuntu 24.04.

2.8 Format of the submitted program

As a reminder, a program has to be submitted only for the qualification and final phases. Teams must provide a zip archive that contains four components:

- A file `Dockerfile`, that will be used to build the container in which the candidate program will run. See Appendix A for examples. Templates are provided on the [challenge gitlab](#).
- A file `run.sh`, that is a wrapper for the candidate program. See Appendix A for examples. Templates are provided on the [challenge gitlab](#).
- Any number of files that implement the candidate program, that will be called by the `run.sh` script.
- A documentation about the solving method

Internet access will be allowed only during the container building phase. After the instances are uploaded in the container, the candidate program is not allowed to use the network, except to contact the Gurobi license server (Cplex and Hexaly licenses are local). The container building phase has the following limitations: the building should not use more than 4GB of RAM and should not last more than half an hour.

The shell script `run.sh` will take 4 arguments:

1. pathname of input file network topology
2. pathname of input file traffic matrix
3. pathname of input file interventions scenario
4. pathname of output solution file srpaths

Example of command line:

```
run.sh toy-net.json toy-tm.json toy-scenario.json toy-srpaths.json
```

The organizing team can provide assistance to help having an up and running Dockerfile and program, provided that there is enough time remaining before the submission deadline.

During the program evaluation, the organizers will run the script `run.sh` **ONCE** for each instance. Ten seconds before the time limit is reached (as a reminder, the time limit is 10mn in the qualification phase and 15mn in the final phase), the script (and all descendant processes) will receive a `SIGTERM` signal, if it is not already finished. The script must then exit gracefully within 10 seconds. After this, it is killed with a `SIGKILL` signal, along with all descendant processes. If the `srpaths` file is not written by then, the solution will be missing and no points will be awarded for this instance.

2.9 Variability

It is the responsibility of participants to support the repeatability of experiments/evaluations. If variability occurs, the organizers can not be responsible from potential bad behavior on this single run.

2.10 Available solvers

The following list provides the commercial solvers available on our evaluation platform:

- CPLEX, version 22.1.2
- GUROBI, version 13.0.2
- HEXALY, version 14.5

2.11 Provided tools

2.11.1 Checker

The provided checker and its source code are available at

<https://gitlab.com/Orange-OpenSource/network-optimization-tools/challenge-roadef-2026>

2.11.2 Networktools

networktools, an internal C++ library developed at Orange Research for modeling and solving network optimization problems (and also used to develop the checker), is available as open source at

<https://gitlab.com/Orange-OpenSource/network-optimization-tools/networktools>

2.12 Contact information

For any inquiries regarding the challenge rules or problem, feel free to contact the organizers at

challenge.roadef2026@orange.com

You can also post an issue on the [challenge gitlab](#)

3 Discussion and updates

To help participants managing their project, the organizers try to communicate immediately changes that might occur during the challenge by updating the dedicated gitlab repository. This includes clarification of the problem description or change in problem instances for example. The organizers try to avoid making such changes unless they find it necessary for the challenge. The organizers reserve the right to disqualify any participant or team from the competition at any time if the participant is considered to have worked outside the spirit of the competition rules.

4 Intellectual property

- Participants have intellectual property on their computer programs developed during the challenge. Orange and any third party may use information provided by the participants through technical reports, scientific papers and oral presentations, but cannot use a computer program off the challenge scope without the author team’s agreement.
- Participants to the challenge cannot claim to have a partnership or a contract with Orange, even if they win the challenge. They can only claim to be participants (respectively qualified / winner) if it is the case.
- Orange may (but has taken no engagement to) sign contracts with some participants after the challenge. Any such contract would be independent of the challenge

Appendix

A Examples of Submissions

A.1 Python Program

If your program is in Python, you can readily use the following `Dockerfile` and `run.sh` files. Please make sure all the required python packages are listed. The two following files must be archived, along with the file `main.py` that implements the candidate resolution algorithm.

Dockerfile

```
#
# Dockerfile template v2 (30/06)
#
FROM ubuntu:24.04
WORKDIR /home
COPY . /home/
RUN chmod -R +x /home/

# -----
# --- modifications start here ---
# -----
# Install python
# Finally, we clean up the apt cache to reduce image size.
RUN apt-get update && apt-get install -y \
    python3 \
    python3-pip \
    python3-venv \
    && rm -rf /var/lib/apt/lists/*

# Install python packages (make sure all required packages are listed)
# You can also use a requirements.txt file (generated with 'pip freeze > requirements.txt')
# by replacing the command below by 'pip install -r requirements.txt'
RUN python3 -m venv /opt/venv
RUN /opt/venv/bin/pip install \
    networkx \
    numpy \
    gurobipy \
    pyomo
ENV PATH="/opt/venv/bin:$PATH"
# -----
# --- modifications end here ---
# -----

RUN chown -R 1006410000:1006410000 /home/
USER 1006410000
CMD ["sleep", "infinity"]
```

run.sh

```
#!/bin/sh
if [ "$#" -ne 4 ]; then
    echo "Usage: $0 <net> <tm> <scenar> <output>"
    exit 1
fi

echo "Starting python program main.py"
python main.py $1 $2 $3 $4
```

A.2 C++ Program with the Networktools library

The following shows the files for a candidate program called `main` that makes use of the Networktools library.

Dockerfile

```
#
# Dockerfile template v2 (30/06)
#
FROM ubuntu:24.04
WORKDIR /home
COPY . /home/
RUN chmod -R +x /home/

# -----
# --- modifications start here ---
# -----
# Install git, and build tools in a single layer
# We add "build-essential" to get g++ and other compilers.
# We also add "git" because it's needed for the "git clone" command.
# Finally, we clean up the apt cache to reduce image size.
RUN apt-get update && apt-get install -y \
    git \
    build-essential \
    && rm -rf /var/lib/apt/lists/*

# Clone a repository and compile the C++ code
RUN git clone
    ↪ https://gitlab.com/Orange-OpenSource/network-optimization-tools/networktools.git
RUN g++ -O3 -std=c++20 -I./networktools/networktools main.cpp -o main
# -----
# --- modifications end here ---
# -----

RUN chown -R 1006410000:1006410000 /home/
USER 1006410000
CMD ["sleep", "infinity"]
```

`run.sh`

```
#!/bin/sh
if [ "$#" -ne 4 ]; then
    echo "Usage: $0 <net> <tm> <scenar> <output>"
    exit 1
fi

echo "Starting c++ program main"
./main $1 $2 $3 $4
```

B How to test your program before submission

When your submission is ready, you can check it will be executed smoothly by the organizing team by following these steps.

1. Store your dockerfile and program in a build directory
2. Build the container from your build directory, and start it
3. Install the instances and, if needed, a solver in the container
4. Run your script `run.sh` on an instance

If your script runs and successfully produces a solution file within the time limit, you are all set to send your submission !

Here is an example of commands used to build the container and start it in an Openshift platform. Please be aware that the actual command you need to enter may differ, depending on your environment.

`Example of commands`

```
oc login
oc new-build --binary --strategy=docker --name=containertest
oc start-build containertest --from-dir=./ --follow
oc run containertest
oc cp setB.tgz containertest:/home/setB.tgz
oc exec containertest -- tar -xzf /home/setB.tgz -C /home/
oc exec containertest -- timeout --kill-after=10 590 ./run.sh setB-01-net.json
→ setB-01-tm.json setB-01-scenario.json setB-01-srpaths.json
```